Brain evolution of a turtle-like robot in an ice surface environment

Thomas Jerver Asmussen, Marcel Zelent

Video: https://youtu.be/SSnOcZQcqXM

1. Introduction

A relatively novel idea in the area of control robotics is using evolutionary algorithms such as Covariance matrix adaptation evolution strategy (CMA-ES) [1] as the brain of a robot. The purpose of the following research was to find how such methods deal with situations, in which there is an absence of clear actuation means, such as legs or wheels. Instead, the robot has to utilize the environment itself as a means of movement. Moreover, the effect of adding sensing capabilities and whether that brings the expected improvement were investigated.

To achieve this, the following setup is proposed; the robot will be a turtle-like creature (size based on the Green Sea Turtle [2]), with only the front arms added. The environment is an ice-like surface, where there is an infinite stretch of a set of pillars, such that the robot is between them and has a straight path ahead of it. The only way to move is by pushing itself away from the pillars. Furthermore, by randomizing the distance between the pillars, it is expected the robot will have to rely on sensors to perform well.

EA: CMA-ES from exercises

World: Custom icy pillar world, turtle-like robot

2. Methods

In this project, the genotype was the weights of a Multi-Layer-Perceptron, while its phenotype is the resulting mapping of observations to torques. The structure of the neural network itself is a simple input layer, one hidden layer and an output layer with a “tanh” activation function each. The weights were constrained to a range between “-1” and “1”. The sizes of the layers are constrained by the way that the simulation is setup; namely the input size is down to the observation space, and output to the action space. In terms of the first one, that was the cartesian position and velocity of the robot. Later on, a set of rangefinders spanning 160 degrees of the front was added. In this way, the “turtle” could see the approaching pillars. Meanwhile, the action space was simply the 4 actuators present on the robot. All was simulated with MuJoCo [3].

The reward function was a matter of discovery. On a high level, the goal was to simply move as far as possible and do that in the shortest possible time. However, as the training of the robot progressed, this was hand-tuned in order to facilitate the desired movement. In the end, an additional term was added to the forward velocity, which took into account the direction in which the robot was facing. Meanwhile, the simulations were terminated based on whether the robot stayed inside the pillar “lane” and if the direction it was facing was still forward. The reward function was:

$$reward = 0.5 \cdot \max(\dot{x}, 0) \cdot \cos(w) + r_{sparse} - 0.5 \cdot |y| - r_{still}$$

Where $\dot{x}$ is the forward speed, y is the horizontal placement, w is the yaw, r_{sparse} is a reward for every 0.5m forward motion, and r_{still} is a punishment for standing still. When including the distance sensors, a "potential" function was added to encourage the robot to move towards the middle of the pillars, by using a weighted sum of the inverse distance activations (normalized between 0 and 1, with one being very close to a pillar), which is weighted towards the sensors pointing to the sides.

3. Results

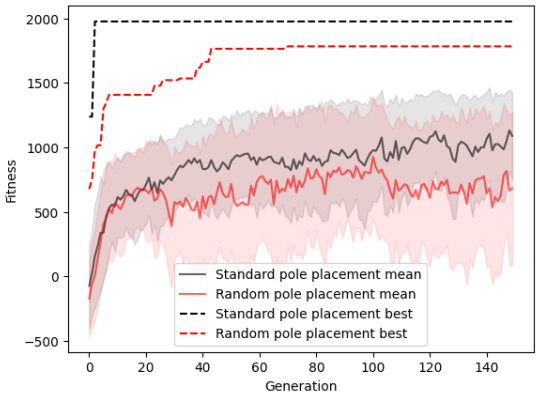

Fig. 1: Performance curve with no sensors

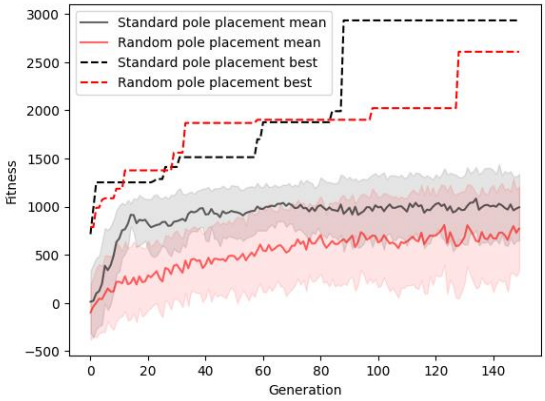

Fig. 2: Performance curve with sensors

In general, the robot has been found to be capable of traversing the environment using only the movement gained from pushing away from the pillars. In many of the simulated versions, it found that the best method to move is by doing a symmetrical movement of the arms away from the pillars, which is what was hypothesized before starting the project.

The plots show the performance with sensors (Fig. 2) and without sensors (Fig. 1). The plots cannot exactly be compared, as when using the sensors the reward function was modified to utilize them. Both versions work well in the standard environment, but though it cannot be seen on the on the figures, the performance improved on the random map when using sensors. Here the algorithm found a reward "hack", where it could avoid the penalty of standing still by going backwards. Overall, as can be seen in the video, the sensors gave a performance boost in both types of environments.

4. Discussion & conclusion

In conclusion, the turtle-like robot learned an intelligent movement method. Adding sensors improved the performance in both environments, but especially when the pillar distances were randomized. For even better performance, the reward function and termination criteria can be further improved, with perhaps even longer episodes of simulation. A bigger next step, would be to change the goal of the environment; from only forward, 1D, traversal to a path in two dimensions. Such a problem would demand much more from the MLP controller.

5. References (IEEE).

[1] N. Hansen and A. Ostermeier, "Adapting arbitrary normal mutation distributions in evolution strategies: the covariance matrix adaptation," *Proceedings of IEEE International Conference on Evolutionary Computation*, Nagoya, Japan, 1996

[2] P. L. Lutz and J. A. Musick, Eds., The Biology of Sea Turtles. Boca Raton, FL, USA: CRC Press, 1997.

[4] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), Portugal, 2012